Table of Contents

Sqlite with Sqlflite **1**
Sqlflite Install 1
Adding Items and Viewing All Items 1
Adding Items with a TextField Input 6
Deleting Items 8
Fetching Item by Id with Parameterized Query 9
Enabling a Detail View 16
Enabling Updates 18
Refreshing the List 22

Submission Instructions

Please do not upload PDF documents for this course going forward. It is okay of course if you already did.

Sqlite with Sqlflite

Sqlflite is the main recommended interface for persistent storage with Sqlite in Flutter.

Sqlflite Install

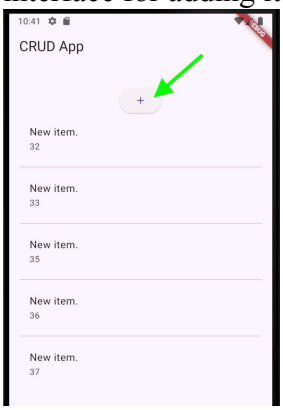
To install Flutter, from terminal run the command:

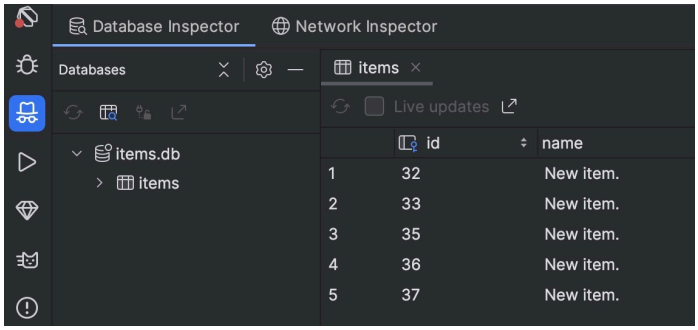
```
flutter pub add sqflite path
```

Adding Items and Viewing All Items

Example 1: Working with Sqlflite

This Sqlflite example shows how to enable database creation in addition to providing a crude interface for adding items and viewing all items. The interface looks like the one shown here:





Clicking the + button on the interface allows you creates a new row entry in the items table of the database. Each row of the table has is given an auto-generated id and a hard-coded name value of 'New Item'. Improvements will be made to make the interface more flexible later.

The **database_helper.dart** file contains a function to build and retrieve the database. Functions for inserting an item, getting all items, updating an item and deleting items are also present here.

lib\database_helper.dart

```dart
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class DatabaseHelper {
  static final DatabaseHelper _instance = DatabaseHelper._internal();
  factory DatabaseHelper() => _instance;
  static Database? _database;

  DatabaseHelper._internal();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDatabase();
    return _database!;
  }

  // Create database if it does not exist.
  Future<Database> _initDatabase() async {
    String path = join(await getDatabasesPath(), 'items.db');
    return await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE items(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT)',
        );
      },
    );
  }

  Future<void> insertItem(String name) async {
    final db = await database;
    await db.insert('items', {'name': name});
  }

  Future<List<Map<String, dynamic>>> fetchItems() async {
    final db = await database;
    return await db.query('items');
  }

  Future<void> updateItem(int id, String name) async {
    final db = await database;
    await db.update('items', {'name': name}, where: 'id = ?', whereArgs: [id]);
  }

  Future<void> deleteItem(int id) async {
    final db = await database;
    await db.delete('items', where: 'id = ?', whereArgs: [id]);
```

The main.dart file builds the interface. A ListView enables scrolling and prevents screen overflow errors.

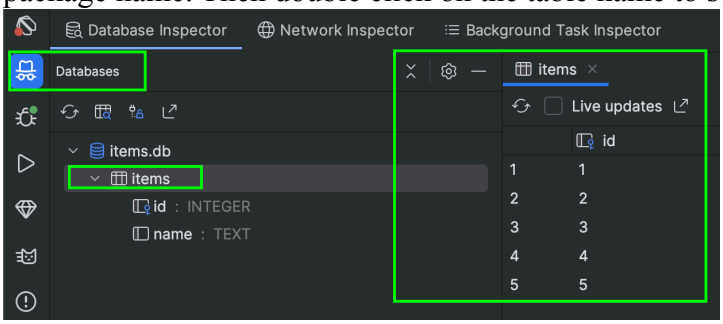
lib\main.dart

```dart
import 'package:flutter/material.dart';
import 'database_helper.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'CRUD App',
      home: MyUiWidget(),
    );
  }
}

class MyUiWidget extends StatefulWidget {
  const MyUiWidget({super.key});

  @override
  MyUiState  createState() => MyUiState ();
}

class MyUiState  extends State<MyUiWidget> {
  final DatabaseHelper dbHelper = DatabaseHelper();
  List<Map<String, dynamic>> _items = [];

  @override
  void initState() {
    super.initState();
    _fetchItems();
  }

  _fetchItems() async {
    final items = await dbHelper.fetchItems();
    setState(() {
      _items = items;
    });
  }

  _addItem() async {
    await dbHelper.insertItem("New item.");
    _fetchItems();
  }
```

When running the code now, an interface like the one shown at the start of the example will appear.

Example 2: Viewing the Database in the App Inspector.

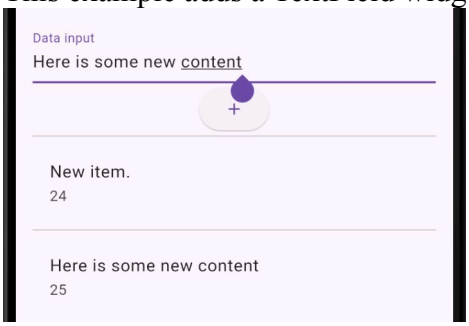
After running the project in Example 1 and after creating several “New Item” entries, in addition to viewing the results on the screen you can view the entries in the App Inspector. To open App Inspector go to **View | Tool Windows | App Inspection**. Then choose your emulator and project package name. Then double click on the table name to see it in the App Inspector.



Adding Items with a TextField Input

Example 3: Enabling a text input

This example adds a TextField widget to Example 1 to allow users to add custom text.



Starting with the MyUiState class from Example 1, add a class level declaration to store the content that is entered into the TextField widget.

```
final TextEditingController _newItemController = TextEditingController();
```

Then, modify the _addItem() function so it can insert the TextField widget content that is submitted into the database.

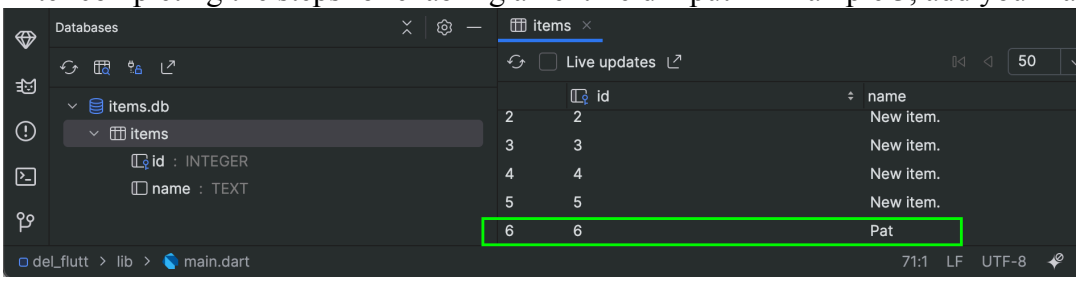
```
_addItem() async {
  await dbHelper.insertItem(_newItemController.text);
  _fetchItems();
}
```

Then, add the following TextField just before the ElevatedButton widget in the build() function:

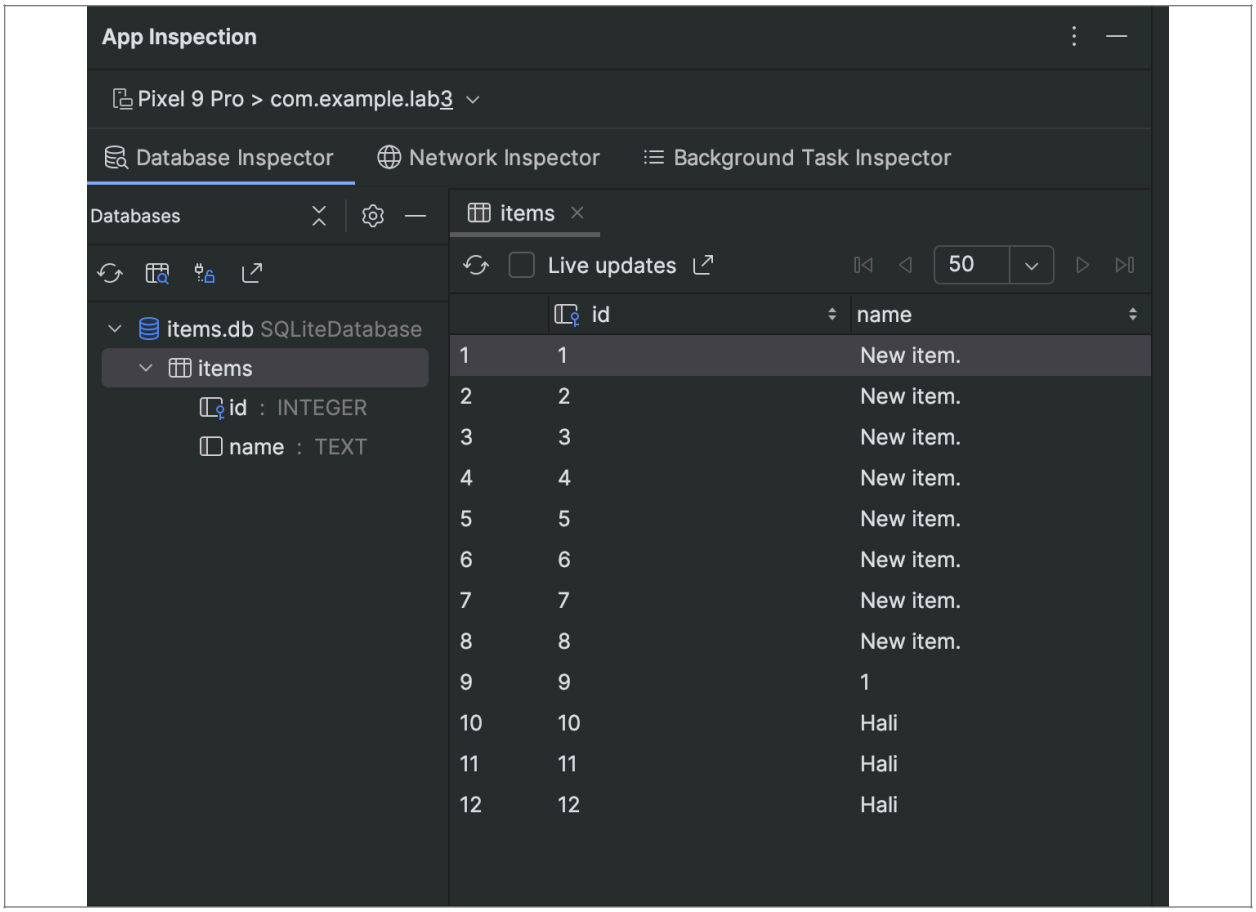
```
TextField(
    controller: _newItemController,
    onChanged: (String text) {
      setState(() {
        _newItemController.text = text;
      });
    },
    decoration: const InputDecoration(
        hintText: "Enter some data",
        labelText: "Data input")),
```

Exercise 1 (2 marks)

After completing the steps for enabling a TextField input in Example 3, add your name.



Show your name in the database in the App Inspector.



Exercise 2 (1 mark)

Use the clear() function of the _newItemController to clear the TextField when the new entry is submitted. Show the code where you add this call to the clear function.

```
_addItem() async {
  await dbHelper.insertItem(_newItemController.text);
  _fetchItems();
  _newItemController.clear();
}
```

Deleting Items

Exercise 3 (2 marks)

In this exercise, you are being asked to enable item deletion. Here is a code shell for the _deleteItem function which belongs in the MyUiState class.

```
_deleteItem(int id) async {
}
```

Here is a code shell for the IconButton which is defined for the **trailing attribute** of the **ListTile** within the **ListView**. Adding this code to the ListTile will add a clickable delete icon for each item in the list.

```
trailing: IconButton(
    icon: Icon(
      Icons.delete,
      color: Colors.grey, // Change color to red to signify deletion
      size: 30.0, // Change size
    ),
   onPressed:() {

   },
),
```

What code must be added to the _deleteItem() function to enable the deletion and refresh the screen?

```
await dbHelper.deleteItem(id);
_fetchItems();
```

What code must be added to the IconButton's onPressed call back?

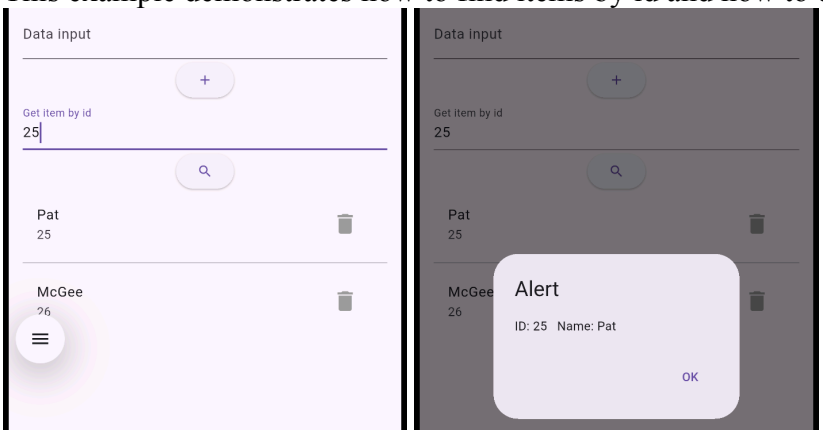
```
_deleteItem(_items[index]['id']);
```

Test the code to be sure the delete function works and to ensure the screen updates accordingly.

Fetching Item by Id with Parameterized Query

Example 4: Fetching items by id

This example demonstrates how to find items by id and how to display them in a basic alert box.



To encapsulate data for each row of the items table, define an Item class that holds the attributes of each row.

```
class Item {
    final int id;
    final String name;

    Item({required this.id, required this.name});
}
```

Next, place the fetchItemById() function in **database_helper.dart** to asynchronously search for a matching item by id. Note that the following declaration defines a immutable dynamic list which contains objects which have any combination of column names and types:

```
final List<Map<String, dynamic>> maps
```

Since only a single object is retrieved, the first attribute is used to limit the query to one row.

```
Future<Item?> fetchItemById(int id) async {
  final db = await database; // Assuming database is already initialized

  // parameterized query to fetch by id.
  // String for column name key and dynamic for any data type (int, String, bool)
  final List<Map<String, dynamic>> maps = await db.query(
    'items',
    where: 'id = ?',
    whereArgs: [id],
  );

  if(maps.isNotEmpty) {
    var itemObj = maps.first;
    return Item(name: itemObj['name'], id: itemObj['id']);
  }
  // Return the first item or null if not found
  return null;
}
```

An alert box will be used to display the item that is found in the database. To store this logic separately, add the showAlert() function to a new file named **alert.dart**.

lib\alert.dart

```
void showAlert(BuildContext context, String alertText) {
  showDialog(
    context: context,
    builder: (BuildContext context) {
      return AlertDialog(
        title: Text("Alert"),
        content: Text(alertText),
        actions: [
          TextButton(
            onPressed: () {
              Navigator.of(context).pop(); // Close the dialog
            },
            child: Text("OK"),
          ),
        ],
      );
    },
  );
}
```

Back in main.dart, declare a controller as a class member of MyUiState to store the string that is entered by the user in the TextField widget.

```
final TextEditingController _itemByIdController = TextEditingController();
```

Next, in **main.dart** add a function to initiate an asynchronous call to fetch the item by id and then display that data in an alert box. Using the build context directly for tracking the widget is not recommended in an async method so a check is made to ensure the widget has been mounted.

```
Future<void> _getItem(BuildContext context) async {
  try {
    int id = int.parse(_itemByIdController.text); // Ensure _idInput is defined
    final Item? item = await dbHelper.fetchItemById(id);

    // Checking if context is mounted during async operations
    // is recommended.
    if (context.mounted && item == null) {
      showAlert(context, "Item is null");
    } else if (context.mounted) {
      showAlert(context, "ID: ${item?.id}   Name: ${item?.name}");
    }
  } catch (e) {
    if (context.mounted) {
      showAlert(context, "Error: $e");
    }
  }
}
```

Exercise 4 (4 marks)

Finally, add the TextField and ElevatedButton widgets to implement the interface for fetching and viewing the data.

Code hints:

- Use the following options in your TextField to ensure only numbers can be entered:

```
    keyboardType: TextInputType.numberWithOptions(
      decimal: false, signed: false),
    inputFormatters: <TextInputFormatter>[
      FilteringTextInputFormatter.allow(RegExp(r'^[1-9][0-9]*$')),
    ],
```

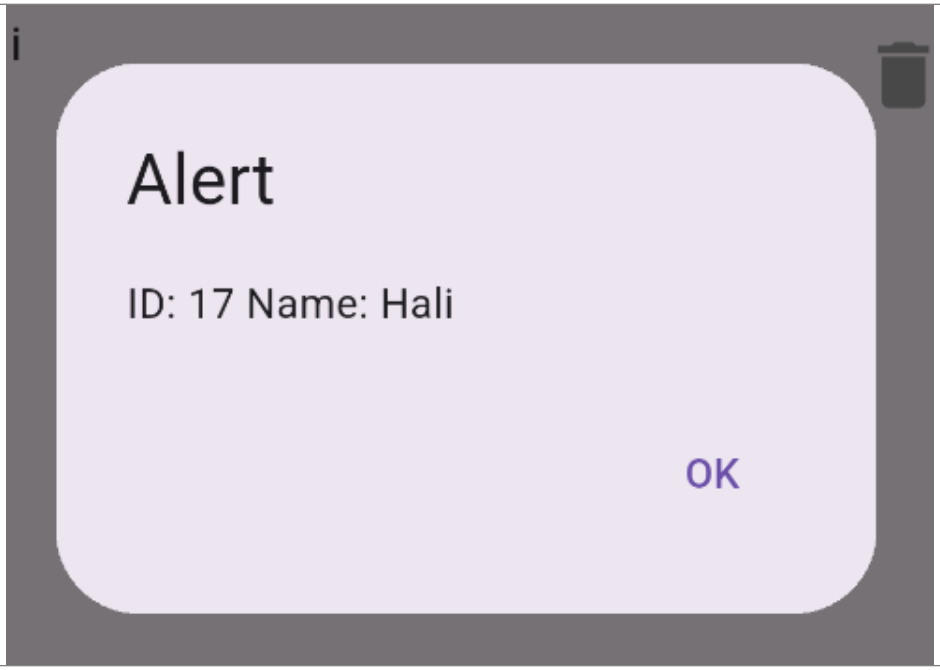
The code for retrieving the item and displaying the result in the Alert dialog looks like the following.

```
Future<void> _getItemById(BuildContext context) async {
  try {
    int id = int.parse(_idController.text); // Adjust controller name as needed.
    _idController.clear()
    final Item? item = await dbHelper.fetchItemById(id);

    // Checking if context is mounted during async operations
    // is recommended to avoid rendering issues.
    if (context.mounted && item == null) {
      showAlert(context, "Item is null");
    } else if (context.mounted) {
      showAlert(context, "ID: ${item?.id}   Name: ${item?.name}");
    }
  } catch (e) {
    if (context.mounted) {
      showAlert(context, "Error: $e");
    }
  }
}
```

The onPressed handler for the new button to submit the id calls the new function like this:

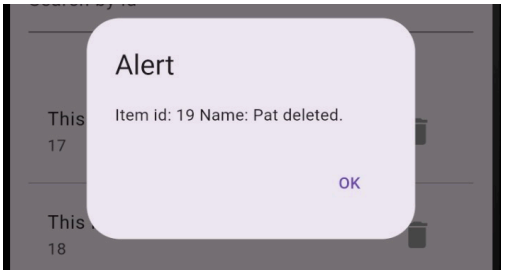
```
onPressed:  () {
  _getItemById(context);
},
```

When finished, take a screenshot of the Alert dialog when it displays an item that was retrieved from the database.



Exercise 5 (2 marks)

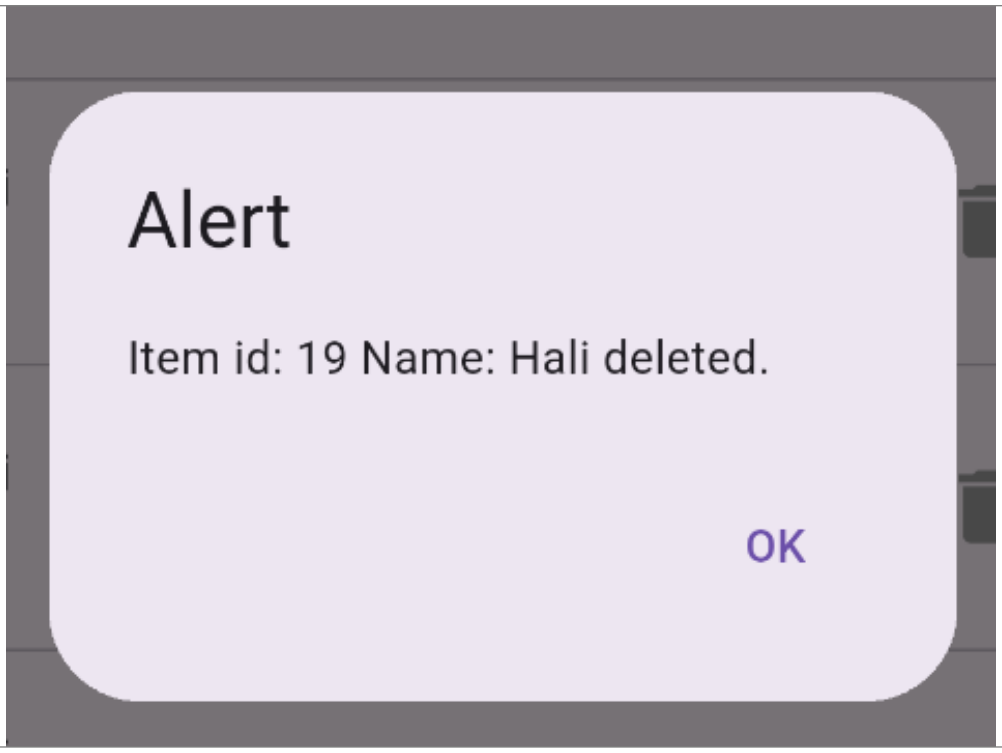
Trigger the existing alert to notify the user when an item has been deleted. It should show the id and name of the item deleted like the following:



For this exercise, you can complete this task inside the onPressed handler for the trailing attribute of the ListTile. Show your updated code here:

```
trailing: IconButton(
  icon: Icon(
    Icons.delete,
    color: Colors.grey,
    size: 30.0,
  ),
    onPressed: () {
      int id = _items[index]['id'];
      String name = _items[index]['name'];
      showAlert(context, "Item id: $id Name: $name
deleted.");
      _deleteItem(id);
    },
},
```

Show a screenshot of the alert box after deleting an item with your name in it.

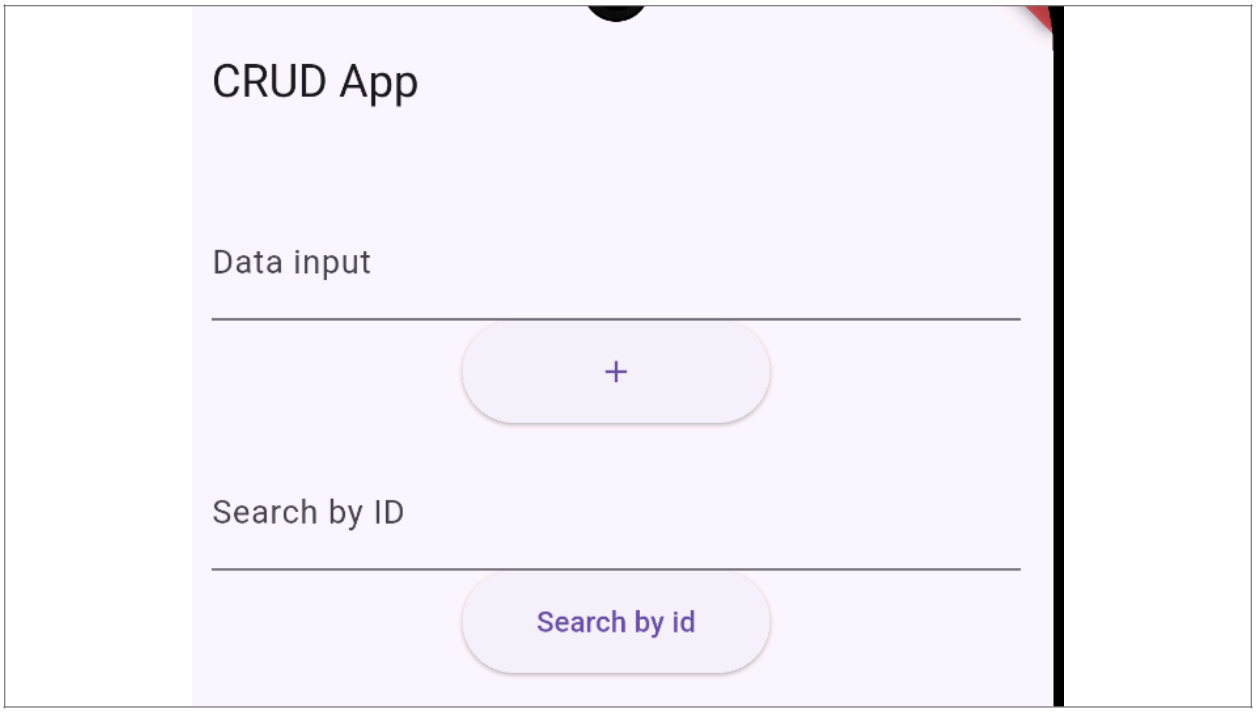


Example 5: Making buttons the same size.

To enable uniform sizing of the add button and the submit search button in Example 4, each can be wrapped in a SizedBox widget which sets the width and height. Here is code which could replace the "**Search by id**" button.

```
SizedBox (
  width: 150.0,
  height: 50.0,
  child: ElevatedButton(
    onPressed: () {
      _getItem(context);
    },
    child: Text("Search by id"),
  ),
),
```

Adjust the button that triggers the addition of new items so it is the same size as the button which submits a search by id. Show a screenshot of your interface once changes have been made to implement identical sizing for each button:



Enabling a Detail View

Example 6: Enabling a Detail View

This example shows how to pass a single object to a detail view when an edit icon is clicked. This was actually covered last day but clarification around variable declaration will be added here. Also, we can take this example further later to enable edits to a single item.

Starting with Example 5, add an edit.dart file and place the following code in it to display the item object *id* and *name* attributes.

lib\edit.dart

```dart
import 'package:flutter/material.dart';
import 'database_helper.dart';

class Edit extends StatefulWidget {
  final Item item;
  Edit({super.key, required this.item}); // Receive & store item object.

  // Not supposed to pass parameters to the
  // State object constructor.
  @override
  MyItemView createState() => MyItemView();
}

class MyItemView extends State<Edit> {
  final DatabaseHelper dbHelper = DatabaseHelper();

  @override
  void initState() {
    super.initState();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
          title: const Text('Edit', style: TextStyle(color: Colors.black)),
          iconTheme: IconThemeData(color: Colors.white),
          leading: IconButton(
            icon: Icon(Icons.arrow_back),
            onPressed: () => Navigator.of(context).pop(false),
            color: Colors.black,
          ),
        ),
        body: Padding(
          padding: const EdgeInsets.all(16.0), // Add padding around the Column

          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: <Widget>[
              Align(
                alignment: Alignment.centerLeft, // Align text to the left
                child: Text("id: ${widget.item.id.toString()}"),
              ),
              Align(
                alignment: Alignment.centerLeft, // Align text to the left
                child: Text("name: ${widget.item.name}"),
              ),
            ],
```

Next, for the ListTile, add this code for the leading attribute to create clickable edit icon which redirects to the detail view.

```
leading: IconButton(
  icon: Icon(
    Icons.edit,
    color: Colors
        .grey, // Change color to red to signify deletion
    size: 30.0, // Change size
  ),
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => Edit(
          item: Item(name: _items[index]['name'], id: _items[index]['id']),
        ),
      ),
    );
  },
),
```

Run the code to be sure that it works.

Exercise 6 (2 marks)

In your own words, explain how the *item* object is referenced from the Stateless widget and stored in the State object within edit.dart.

When navigating to the Edit screen, the Item object is passed to the Edit StatefulWidget through its constructor and stored as a final variable. Since the State class cannot receive parameters directly, it accesses the Item using widget.item. Therefore, item properties are accessed as widget.item.id or widget.item.name.

Enabling Updates

To enable updates to the text, inside the state object within edit.dart, add a declaration for the TextEditingController which can be used with the TextField widget.

```
final TextEditingController _controller = TextEditingController();
```

Then replace the Column widget inside **lib\edit.dart** with the following code:

```
Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: <Widget>[
    Align(
      alignment: Alignment.centerLeft, // Align text to the left
      child: Text("id: ${widget.item.id}"),
    ),
    TextField(
        controller: _controller,
        onChanged: (String text) {
          setState(() {

          });
        },
        decoration: const InputDecoration(
            hintText: "Edit item name", labelText: "Edit item name")),
    Row(
      mainAxisAlignment:
      MainAxisAlignment.center, // Center the buttons
      children: <Widget>[
        ElevatedButton(
          onPressed: () {
          },
          child: Text('Submit'),
        ),
        SizedBox(width: 20), // Add space between buttons

      ],
    ),
  ],
),
```

Also inside the state object, add this function to update the *item* object with any changes to the *name* property and then to navigate back to the original screen.

```
_updateItems() async {
  var localContext = context;
  await dbHelper.updateItem(widget.item.id, _controller.text);
  if(localContext.mounted) { //<-Recommended approach when using context with async.
    Navigator.of(localContext).pop(false);
  }
}
```

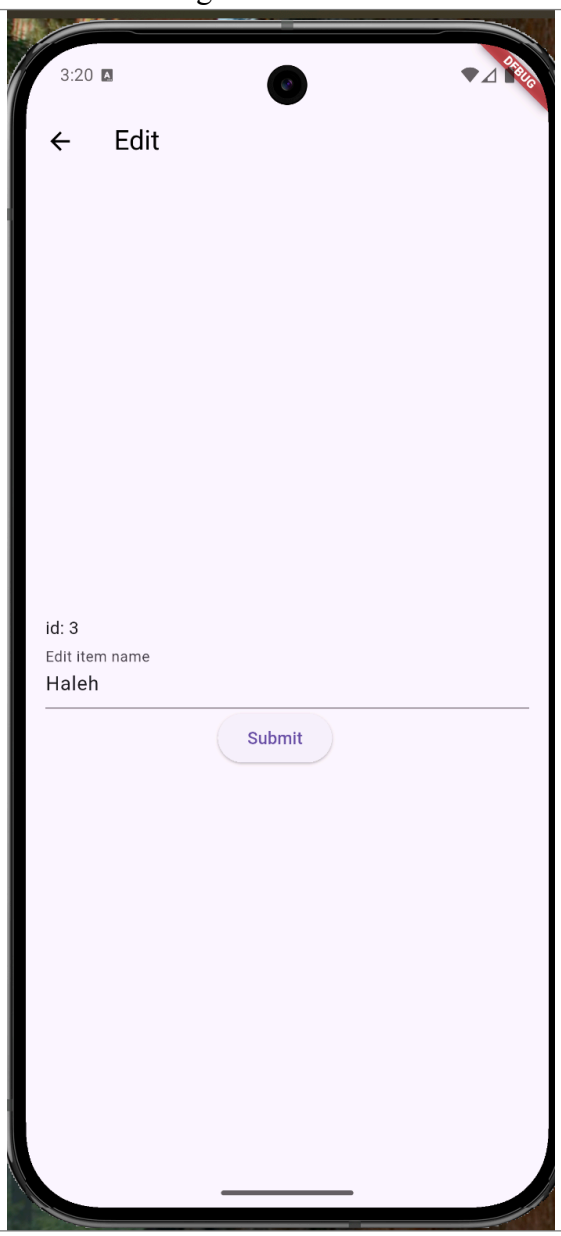
Exercise 7 (1 mark)

Where does _updateItems() need to be called in **lib\edit.dart**? Be sure to implement this.

Inside onPressed handler for the submit button.

Exercise 8 (1 mark)

What assignment to **_controller.text** property needs to occur for the existing item.name to appear in the TextField when the form appears? Where does this assignment need to take place? Be sure to implement this and also check it. Show a screenshot of the screen which shows the original name in the text box when arriving in the edit screen.



Show the code and enclosing method required to do this:

```
@override
void initState() {
  super.initState();
  _controller.text = widget.item.name;
}
```

Exercise 9 (1 mark)

What assignment to **_controller.text** property needs to occur when the TextField content changes? Where does this assignment need to take place? Be sure to implement this.

The assignment _controller.text = text; needs to be placed inside the onChanged callback of the TextField.

Test your work to ensure you can change the text and submit it.

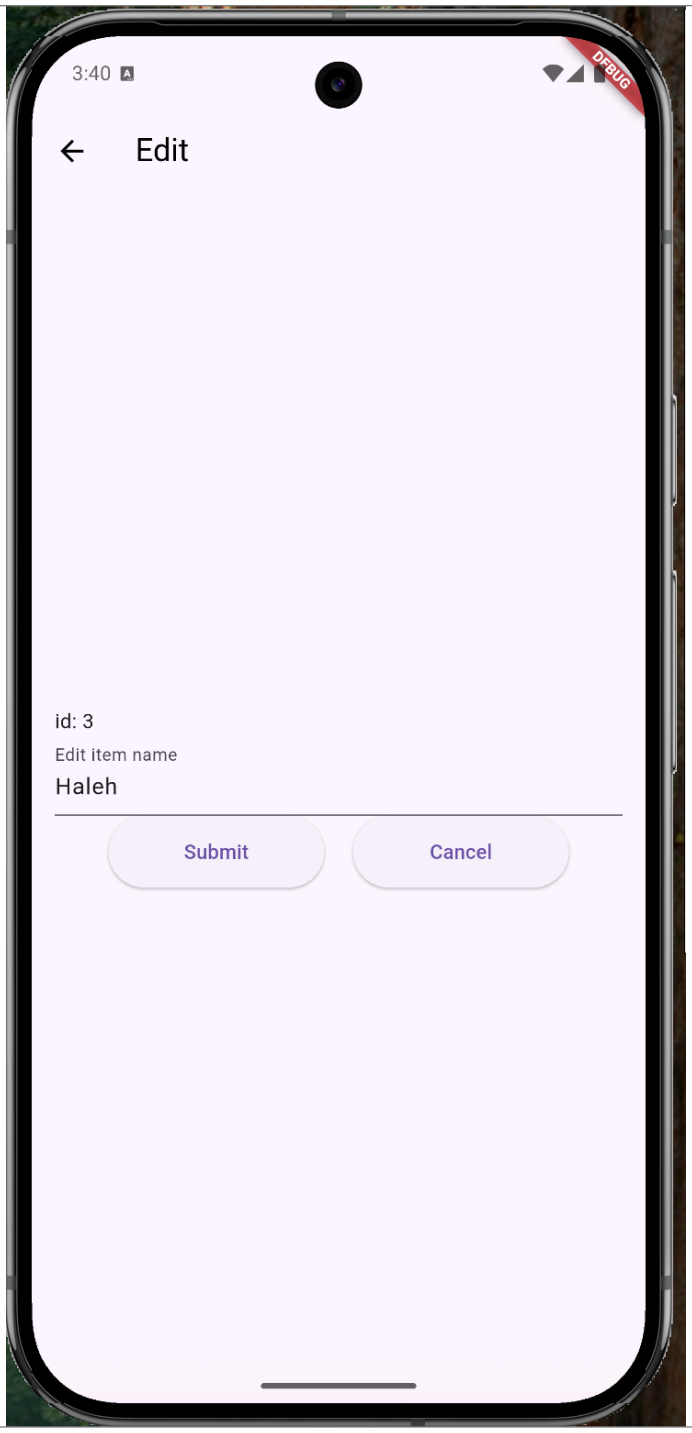
Note: The listing in main.dart does not show any edits that are submitted until the app is stopped and restarted. We will fix this soon.

Exercise 10 (2 marks)

In edit.dart, add a cancel button beside the submit button that abandons all edits and returns to the main screen. Show your code for the cancel button here:

```
ElevatedButton(
  onPressed: () {
    Navigator.of(context).pop(false);
  },
  child: Text('Cancel'),
),
```

Show a screenshot of the edit screen with the submit and cancel buttons side by side. See Example 5 for how to size the buttons identically.



Refreshing the List

As mentioned earlier, the listing in main.dart is not updating when edits are submitted. One way to fix this issue involves sending a reference to the _fetchItems() function from the main screen to the edit screen. In this way, the child can invoke the refresh when navigating back.

First in lib/edit.dart make adjustments to receive and store a reference to the function.

```
class Edit extends StatefulWidget {
  final Item item;
  final Function() refreshParentScreen; // Store reference to call back.
  const Edit({super.key, required this.item, required this.refreshParentScreen});
}
```

Then inside _updateItems() add the instruction to invoke the callback.

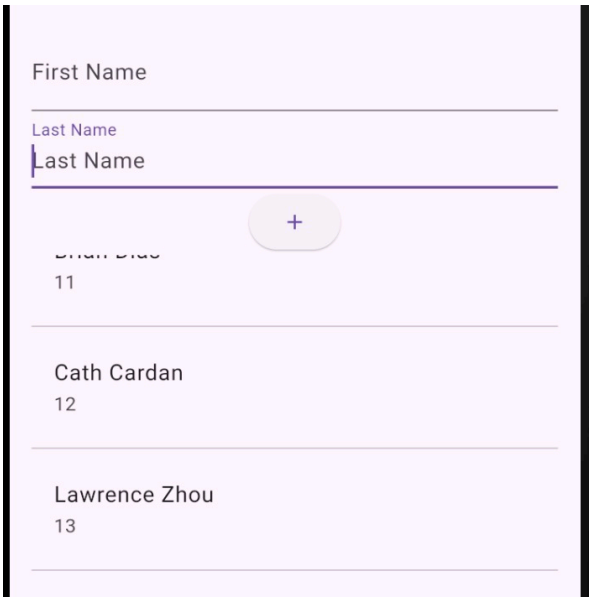
```
  _updateItems() async {
    var localContext = context;
    await dbHelper.updateItem(item.id, _nameInput);
    if(localContext.mounted) {
      Navigator.of(localContext).pop(false);
      widget.refreshParentScreen();
    }
  }
```

Finally inside **main.dart**, add code to pass _fetchItems to the edit screen.

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => Edit(
      item: Item(
          name: _items[index]['name'],
          id: _items[index]['id']),
          refreshParentScreen: _fetchItems,
    ),
  ),
);
```

Exercise 11 (8 marks)

Starting with Example 1, inside the database_helper.dart file, rename the database to ‘people_v1.db’. Then adjust the data table so it stores fname, lname and id. Make appropriate adjustments to the query that selects all items and to the query that inserts a record. Then implement the following interface below. You are not required to perform delete, insert or updates.



Show your code for your database_helper.dart file here: I put whole code including delete/update as well.

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';


class Item {
  final int id;
  final String name;

  Item({required this.id, required this.name});
}

class DatabaseHelper {
  static final DatabaseHelper _instance = DatabaseHelper._internal();
  factory DatabaseHelper() => _instance;
  static Database? _database;

  DatabaseHelper._internal();

  Future<Database> get database async {
    if (_database != null) return _database!;
```

```
    _database = await _initDatabase();
    return _database!;
  }

  // Create database if it does not exist.
  Future<Database> _initDatabase() async {
    String path = join(await getDatabasesPath(), 'people_v1.db');
    return await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE items(id INTEGER PRIMARY KEY AUTOINCREMENT, fname TEXT, lname TEXT)',
        );
      },
    );
  }

  Future<void> insertItem(String fname, String lname) async {
    final db = await database;
    await db.insert('items', {'fname': fname, 'lname': lname});
  }

  Future<List<Map<String, dynamic>>> fetchItems() async {
    final db = await database;
    return await db.query('items');
  }

  Future<void> updateItem(int id, String name) async {
    final db = await database;
    await db.update('items', {'name': name}, where: 'id = ?', whereArgs: [id]);
  }

  Future<void> deleteItem(int id) async {
    final db = await database;
    await db.delete('items', where: 'id = ?', whereArgs: [id]);
  }
```

```
  Future<Item?> fetchItemById(int id) async {
    final db = await database; // Assuming database is already initialized

    // parameterized query to fetch by id.
    // String for column name key and dynamic for any data type (int, String, bool)
    final List<Map<String, dynamic>> maps = await db.query(
      'items',
      where: 'id = ?',
      whereArgs: [id],
    );

    if(maps.isNotEmpty) {
      var itemObj = maps.first;
      return Item(name: itemObj['name'], id: itemObj['id']);
    }
    // Return the first item or null if not found
    return null;
  }
}
```

Show your main.dart file here:

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'database_helper.dart';
import 'alert.dart';
import 'edit.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  MyApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'CRUD App',
      home: MyUiWidget(),
    );
  }
}

class MyUiWidget extends StatefulWidget {
  const MyUiWidget({super.key});

  @override
  MyUiState createState() => MyUiState();
}

class MyUiState extends State<MyUiWidget> {
  final DatabaseHelper dbHelper = DatabaseHelper();
  List<Map<String, dynamic>> _items = [];
  final TextEditingController _fnameController = TextEditingController();
  final TextEditingController _lnameController = TextEditingController();
  final TextEditingController _idController = TextEditingController();

  @override
  void initState() {
    super.initState();
    _fetchItems();
  }

  _fetchItems() async {
    final items = await dbHelper.fetchItems();
    setState(() {
      _items = items;
    });
  }

  _addItem() async {
    await dbHelper.insertItem(_fnameController.text, _lnameController.text);
    _fetchItems();
    _fnameController.clear();
```

```
    _lnameController.clear();
  }

  _deleteItem(int id) async {
    await dbHelper.deleteItem(id);
    _fetchItems();
  }

  Future<void> _getItemById(BuildContext context) async {
    try {
      int id = int.parse(_idController.text); // Adjust controller name as needed.
      _idController.clear();

      final Item? item = await dbHelper.fetchItemById(id);

      // Checking if context is mounted during async operations
      // is recommended to avoid rendering issues.
      if (context.mounted && item == null) {
        showAlert(context, "Item is null");
      } else if (context.mounted) {
        showAlert(context, "ID: ${item?.id} Name: ${item?.name}");
      }
    } catch (e) {
      if (context.mounted) {
        showAlert(context, "Error: $e");
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('CRUD App'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: <Widget>[
            SizedBox(height: 16.0),
```

```
          TextField(
            controller: _fnameController,
            decoration: const InputDecoration(
              hintText: "Enter first name",
              labelText: "First Name",
            ),
          ),
          SizedBox(height: 8.0),
          TextField(
            controller: _lnameController,
            decoration: const InputDecoration(
              hintText: "Enter last name",
              labelText: "Last Name",
            ),
          ),
          SizedBox(
            width: 150.0,
            height: 50.0,
            child: ElevatedButton(
              onPressed: _addItem,
              child: Icon(Icons.add),
            ),
          ),
          SizedBox(height: 16.0),
          TextField(
            controller: _idController,
            keyboardType: TextInputType.numberWithOptions(decimal: false, signed: false),
            inputFormatters: <TextInputFormatter>[
              FilteringTextInputFormatter.allow(RegExp(r'^[1-9][0-9]*$')),
            ],
            decoration: const InputDecoration(
              hintText: "Enter ID",
              labelText: "Search by ID",
            ),
          ),
          SizedBox(
            width: 150.0,
            height: 50.0,
            child: ElevatedButton(
              onPressed: () {
```

```
                _getItemById(context);
              },
              child: Text("Search by id"),
            ),
          ),
          Expanded(
            child: ListView.separated(
              itemCount: _items.length,
              itemBuilder: (context, index) {
                return ListTile(
                  title: Text('${_items[index]['fname']} ${_items[index]['lname']}'),
                  subtitle: Text(_items[index]['id'].toString()),
                  leading: IconButton(
                    icon: Icon(
                      Icons.edit,
                      color: Colors
                          .red, // Change color to red to signify deletion
                      size: 30.0, // Change size
                    ),
                    onPressed: () {
                      Navigator.push(
                        context,
                        MaterialPageRoute(
                          builder: (context) => Edit(
                            item: Item(
                                name: _items[index]['name'],
                                id: _items[index]['id']),
                            refreshParentScreen: _fetchItems,
                          ),
                        ),
                      );
                    },
                  ),
                  trailing: IconButton(
                    icon: Icon(
                      Icons.delete,
                      color: Colors.grey,
                      size: 30.0,
                    ),
                      onPressed: () {
```

```
                        int id = _items[index]['id'];
                        String name = _items[index]['name'];
                        showAlert(context, "Item id: $id Name: $name deleted.");
                        _deleteItem(id);
                      },
                    ),
                  );
                },
                separatorBuilder: (context, index) => Divider(),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```